

Software / System Deconstruction using UML Models

A Business Analyst's Report on the Servo Saver Fuel-Price Feature of the
Service Victoria App

Individual Project Report

Course: ISYS2395 — Business Process and Systems Analysis

Student: R7amos | Student ID: [insert ID]

Tutor: [insert tutor] | RMIT University

June 2026

Table of Contents

1. Introduction and Scope.....	1
2. Business Case.....	2
3. Activity Diagram: Searching for Nearby Fuel.....	4
4. Overall Use Case Diagram.....	7
5. Detailed (Expanded) Use Case: Search Nearby Fuel Stations.....	10
6. Class Diagram.....	13
7. State Machine Diagram: the FuelPrice Object.....	17
8. User Interface Design: the Good, the Bad and the Ugly.....	20
9. Proposed Improvement: Price Drop Alerts.....	22
10. Incorporating the Improvement into the Models.....	26
11. Limitations and Future Work.....	35
12. Conclusion.....	35
References.....	35
Appendix.....	36

List of Figures

Figure 3.1 — Activity diagram: searching for nearby fuel

Figure 4.1 — Overall use case diagram (current system)

Figure 6.1 — Class diagram (current system)

Figure 7.1 — State machine diagram: the FuelPrice object

Figure 8.1–8.4 — Current Servo Saver interface (captured screens)

Figure 9.1 — Improved station-detail screen (proposed)

Figure 10.1 — Improved overall use case diagram

Figure 10.3 — Improved class diagram

Tables

Table 2.1 — Australian fuel-retailing and Servo Saver: key figures

Table 5.1 — Expanded use case description: UC1 Search nearby fuel stations

Table 6.1 — CRC card: FuelStation

Table 6.2 — CRC card: FuelPrice

Table 9.1 — Value drivers of Price Drop Alerts and supporting evidence

Table 10.1 — Expanded use case: Set price drop alert (new)

Table 10.2 — CRC card: PriceAlert (new class)

Table 10.3 — CRC card: AlertNotification (new class)

Executive Summary

This report presents an information-systems business analyst's deconstruction of Servo Saver, the fuel-price comparison feature built into the Victorian Government's Service Victoria mobile application, and proposes a single, well-evidenced improvement, modelled in UML, that can be delivered without disrupting the existing system. It is written for the system's owner and explains every diagram in plain language before analysing it.

Servo Saver lets Victorian motorists compare government-reported fuel prices at nearby service stations on a familiar map or ranked list, highlights the cheapest options, and shows each retailer's locked maximum price for the following day — letting a driver decide whether to fill up today or wait. In a market where prices move daily through sharp cycles, this transparency is a genuine public good.

The analysis finds that, for all its strengths, the feature is entirely "pull-based": its value depends on the motorist remembering to open the app and look at exactly the right moment. Trust in the figures is further weakened by a single, app-wide "updated" timestamp rather than a per-station freshness indicator, and by presentation defects such as a prominent "opening hours not available" line and the absence of any way to save a station or build a watchlist. These are problems of reach and trust rather than of data quality.

These conclusions were reached through a structured, hands-on walk-through of the live application, supported by annotated screen captures, and a full UML deconstruction of the system across activity, use case, class and state-machine models, each narrated so that a non-technical reader can follow it.

To address the core weakness, the report proposes a Price Drop Alerts feature: a registered account holder sets a target price, fuel type, search radius and optional quiet hours, and the system proactively notifies them when a nearby station falls to or below the target. Because it reuses components the platform already holds — location, the price records, the registered account and the locked daily price cap — it can warn users the evening before a price change at minimal cost, and, on published evidence that proactive notifications lift app retention and that well-timed purchases save money, it is expected to raise engagement, retention and motorist savings.

For clarity and depth within a single-author scope, this report analyses the system from the customer (motorist) perspective only; the retailer, administrator and regulator roles appear for context and are revisited in Limitations.

1. Introduction and Scope

This report examines Servo Saver, the fuel-price comparison feature built into the Service Victoria app, from the perspective of an information systems (IS) business analyst — remembering that IS combines information technology with the business it serves. The aim is twofold: first, to analyse the existing system comprehensively using a set of Unified Modelling Language (UML) diagrams so that a non-technical reader can understand how the system behaves; and second, to identify a worthwhile improvement and show, again through UML models, exactly how that improvement would change the existing design and add value.

Servo Saver is a real, recently launched government service. Because the full system serves several very different groups — motorists who look up prices, fuel retailers who submit prices, Service Victoria administrators who govern the platform, and Consumer Affairs Victoria which enforces compliance — a single-author report cannot do justice to every perspective at the depth required. The scope of this report is therefore deliberately limited to the customer (motorist) perspective. The retailer, administrator and regulator roles are included in the models only where they touch the motorist's experience, and the report is candid about this boundary in the Limitations section.

The report is organised to follow the assessment structure. It opens with a business case that sets the industry and company context with supporting statistics. It then presents and explains, one at a time, an activity diagram, an overall use case diagram, a detailed (expanded) use case, a class diagram and a state-machine diagram. Each diagram is introduced in plain language, its notation is explained, and then it is narrated element by element so that a reader who has never seen UML can still follow it. The report then evaluates the user interface, proposes the Price Drop Alerts improvement with evidence of its value, and finally shows the improvement incorporated into revised use case and class diagrams with every new element explained in detail.

1.1 Method of analysis

The analysis combined three sources of evidence. First, the live Servo Saver feature was examined hands-on through a structured walk-through of its main screens — the map view, the ranked list, the fuel-type selector, the brand filter and a station-detail screen — and these screens were captured to ground the interface evaluation in what the system actually shows the user rather than in assumption. Second, the public record of Victoria's fuel reforms and the Service Victoria platform was reviewed to establish the industry and policy context and to confirm how the daily price-cap mechanism operates. Third, the system's structure and behaviour were

reconstructed as UML models from this observed behaviour. Because the analysis works from the outside in — from what the system does, to how it must be structured to do it — the models are a faithful interpretation of a real service rather than a copy of any internal design document, and the report is explicit wherever a modelling assumption has been made.

2. Business Case

2.1 The industry context

Fuel retailing is one of the largest consumer-facing industries in Australia. Industry analysis by IBISWorld (2025) estimates that revenue in the Australian fuel-retailing industry reaches approximately \$58.7 billion, having grown at an annualised rate of about 4.4% over the five years to 2024–25. Fuel is a near-unavoidable cost for households and businesses, yet its price is unusually volatile: in capital cities the bowser price moves through regular "price cycles" in which it climbs sharply and then drifts down over a number of days. The Australian Competition and Consumer Commission (ACCC, 2024) reports that a Melbourne motorist who consistently buys at the low point of these cycles, rather than the high point, could save in the order of \$333 a year. The difficulty for ordinary consumers has always been knowing where, and when, the cheapest fuel can be found — which is precisely the gap a price-transparency service is designed to close.

Australia has progressively addressed this gap through mandated price-reporting schemes. Western Australia's FuelWatch has operated since 2001, and New South Wales and other jurisdictions followed with real-time "fuel check" services (ACCC, 2024). Victoria was the last mainland state to act, introducing mandatory real-time price reporting and, in a second phase, a daily price-cap rule under its Fair Fuel Plan (Premier of Victoria, 2025). These reforms make accurate, timely price information a public good — and a digital channel through which to deliver it becomes essential infrastructure rather than a convenience.

Figure	Source
Australian fuel-retailing revenue (2024–25): $\approx$ \$58.7 billion	IBISWorld (2025)
Five-year annualised industry growth to 2024–25: $\approx$ 4.4%	IBISWorld (2025)
Potential annual saving for a Melbourne motorist buying at the cycle low: $\approx$ \$333	ACCC (2024)
Servo Saver launch: October 2025	Premier of Victoria (2025)
Daily price cap: submitted by 2pm, published 4pm, locks from 6am for 24h	Premier of Victoria (2025)
Penalty per reporting breach: over \$3,000 (serious offences over \$24,000)	Consumer Affairs Victoria (2025)

Figure	Source
Services hosted on the Service Victoria platform: 100+	Service Victoria (2025)

Table 2.1. Key figures for the Australian fuel-retailing industry and the Servo Saver service, with sources.

2.2 The company and the system

Service Victoria is the Victorian Government's central digital-services platform. The Victorian Auditor-General's Office (VAGO, 2020) examined Service Victoria as the state's vehicle for digital delivery of government transactions, and the platform has since expanded to host more than one hundred services — from vehicle registration and Working with Children Checks to digital licences and concession cards (Service Victoria, 2025). Delivering the fuel-price service inside this established "super-app" is significant: it means motorists do not need to create a new account or download a separate application, and the service inherits the platform's identity, security and trust.

Servo Saver itself was launched in October 2025 as the consumer face of Victoria's fuel reforms (Premier of Victoria, 2025). Under the daily price-cap rule, every retailer must submit its maximum price for each fuel grade by 2pm; that price is published on Servo Saver by 4pm and then locks in from 6am the next morning for a full 24 hours (Premier of Victoria, 2025). Retailers that fail to register or report correctly face infringement notices from Consumer Affairs Victoria, with penalties exceeding \$3,000 per breach and more serious offences attracting court action and penalties above \$24,000 (Consumer Affairs Victoria, 2025). The system therefore exists to serve a clear public purpose: to make fuel prices transparent and predictable, to dampen the mid-cycle price spikes that catch drivers off guard, and to support enforcement of the new rules.

2.3 Services and problems the system addresses

For the motorist, Servo Saver answers two practical questions. The first is "where is the cheapest fuel near me right now?", which it answers by plotting current, government-reported prices on a familiar map and flagging the lowest. The second, less common but valuable, is "should I fill up today or wait?", which it supports by publishing tomorrow's locked price cap alongside today's price. By turning a previously opaque market into a transparent one, the service reduces the search cost that consumers face and applies competitive pressure on retailers to price keenly.

The problems that remain are problems of reach and trust rather than of data. Because the service is something the motorist must actively open and inspect, its benefit depends entirely on the user remembering to check at the right moment — the very thing busy drivers do not do. And because price freshness is communicated through a single app-wide timestamp, a motorist cannot easily judge how current any one station's figure is. These are the opportunities the proposed improvement targets.

2.4 Opportunities

The opportunity, in business terms, is to convert a passive lookup tool into a proactive assistant. A government service that actively helps citizens save money at a time of cost-of-living pressure earns goodwill, drives repeat engagement with the wider Service Victoria platform, and amplifies the policy intent of the fuel reforms by steering demand toward the cheapest, most competitively priced stations. Adjacent opportunities include personalisation (favourite stations and routes), broader coverage of fuel types, and, over a longer horizon, electric-vehicle charging prices. The Price Drop Alerts feature proposed in this report is the most direct and lowest-cost of these opportunities because it reuses data the system already collects.

3. Activity Diagram: Searching for Nearby Fuel

3.1 What this diagram is and why it exists

An activity diagram is, in effect, a structured flowchart. It shows the step-by-step flow of a single business process from start to finish, including the points where the process makes a decision and branches, and the points where two things happen at the same time (in parallel). It is the right tool for explaining "how does this actually work, step by step?" to someone who does not need to see the underlying code. The process modelled here is the core motorist journey: opening the app and finding nearby fuel prices, labelled UC1 — Search nearby fuel stations.

3.2 How to read the notation

Before reading the diagram, five symbols are worth knowing. A solid black circle is the initial node — where the process starts. A rounded rectangle is an action — a single step of work. A diamond is a decision node: the flow leaves it along whichever path's guard condition, shown in square brackets like [GPS permission granted], is true. A solid black bar is a fork or a join: a fork splits the flow into parallel paths that run at the same time, and a join waits for those parallel paths to finish before continuing. The bordered circle at the end is the final node. The diagram also uses vertical "swimlanes", one per participant, so the reader can see who or what performs each step.

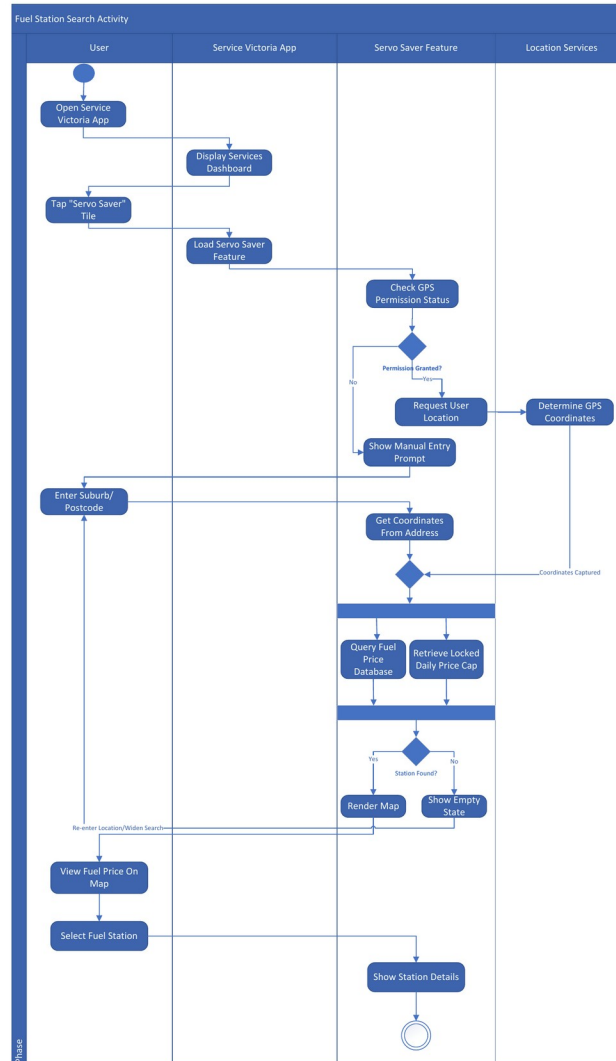


Figure 3.1. Activity diagram for UC1 — the motorist searches for nearby fuel. Swimlanes separate the motorist, the Service Victoria host app, the Servo Saver module and Location Services. A decision node handles GPS permission, and a fork/join runs the price query and price-cap retrieval in parallel.

3.3 Element-by-element walk-through

The process begins in the motorist's swimlane when the user opens the Service Victoria app, which responds by displaying its services dashboard. The motorist taps the "Servo Saver" tile,

and the Servo Saver module loads and checks the device's GPS permission status. This check is the first decision node. If permission has been granted, the flow follows the [GPS permission granted] path: the module requests the user's location and Location Services determines the GPS coordinates, which return to the module as a captured "Coordinates" object. If permission has not been granted, the flow follows the [no permission] path instead: the module shows a manual-entry prompt, the motorist enters a suburb or postcode, and the module geocodes that text into coordinates. Either way, the process arrives at the same point with a usable location — a good example of a decision node bringing two alternative paths back together.

At this point the diagram shows a fork. Two activities run in parallel: the module queries the fuel-price database for current prices, and it retrieves the locked daily price caps. Modelling these as parallel rather than sequential reflects how the system actually behaves — neither task depends on the other, so both are launched together and a join waits for both to complete before the process continues. This is exactly the kind of parallelism, decision-making and guarding that distinguishes a considered activity diagram from a simple linear list of steps.

After the join, a second decision evaluates the result. On the [stations found] path the module renders the map of nearby stations with their prices; on the [none found] path it shows an empty state and offers the motorist the chance to re-enter the location or widen the search, looping back into the flow. With results displayed, the motorist views the fuel prices on the map and selects a fuel station, and the module shows that station's details. The process then reaches its final node.

3.4 Relationships and logic

The logic of the diagram is driven by two questions and one parallel step. The first decision (GPS permission) exists because location is essential but cannot be assumed; the manual-entry branch is the system's graceful fallback. The fork/join exists because retrieving prices and retrieving caps are independent, so doing them together makes the experience faster. The second decision (stations found or not) protects the user from a dead end by offering a recovery path. Reading the swimlanes from left to right also makes the division of labour explicit: the motorist initiates and chooses, the host app and the Servo Saver module coordinate, and Location Services supplies the coordinates.

3.5 Business meaning

For a decision-maker, this diagram demonstrates that the headline motorist journey is short, resilient and fast: it copes with denied GPS permission, it parallelises work to keep the map responsive, and it never leaves the user stranded when no results are found. It also exposes

where the journey ends — at "view prices" and "select station" — which is the natural point at which the proposed alerts feature later extends the experience beyond a single look-up.

4. Overall Use Case Diagram

4.1 What this diagram is and why it exists

A use case diagram answers a different question from the activity diagram. Rather than showing the steps of one process, it gives a bird's-eye view of all the services the system offers and of who uses each service. The stick figures are actors — the people or external systems that interact with the software — and the ovals are use cases — the discrete services the system provides. It is the ideal diagram for the business reader because it summarises, on one page, everything the system does and for whom.

4.2 How to read the notation

A plain line between an actor and a use case is an association: it simply means that actor takes part in that service. Two dotted-arrow relationships carry special meaning. An «include» arrow means one use case always uses another as part of its job — for example, searching always includes detecting the user's location. An «extend» arrow means one use case sometimes adds optional behaviour to another — for example, filtering results is an optional extension of displaying them. A triangle-headed line is a generalisation, meaning one actor or use case is a specialised kind of another. The boxed region is the system boundary: everything inside it is the responsibility of Servo Saver.

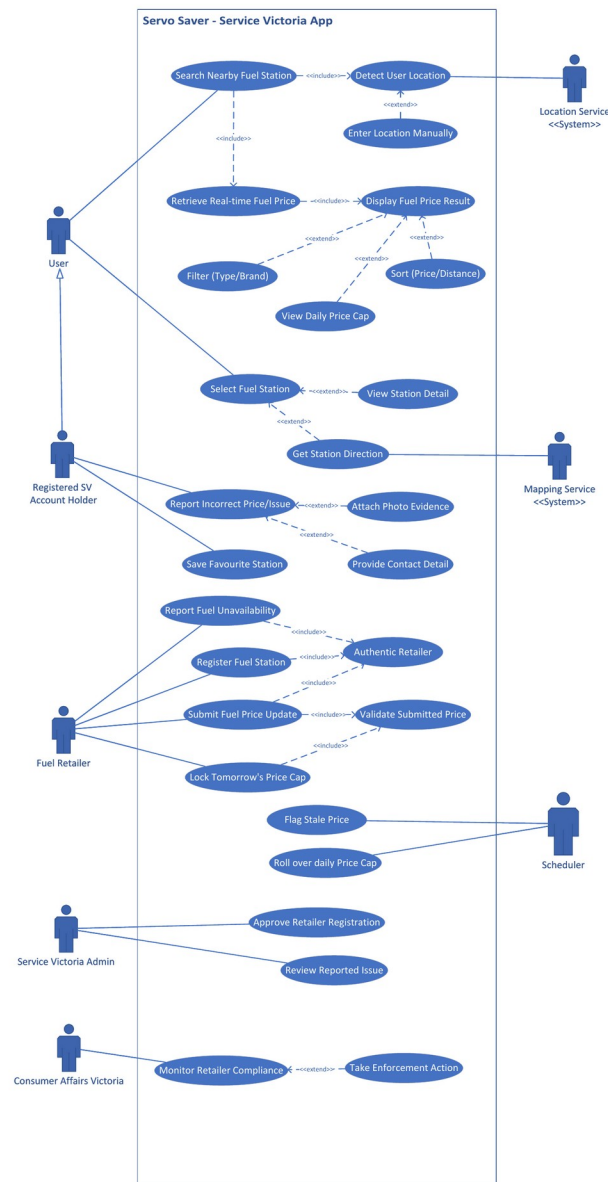


Figure 4.1. Overall use case diagram for the current Servo Saver system, grouped into motorist, fuel-retailer, scheduled-system, administrator and regulator flows. The Registered SV Account Holder is shown as a specialisation of the User (motorist) actor.

4.3 Element-by-element walk-through

The diagram groups the services into logical lanes. In the motorist lane, the central service is Search nearby fuel stations. This hub use case «includes» three services it always needs: Detect user location, Retrieve real-time fuel prices and Display fuel price results. Around it sit optional extensions, each joined by an «extend» arrow: Enter location manually (used only when GPS is unavailable), Filter (type / brand), Sort (price / distance) and View daily price cap (tomorrow). Beyond searching, the motorist can Select fuel station, which leads to View station details, Get directions to station and Save favourite stations; and the motorist can Report incorrect price / issue, which is optionally extended by Attach photo evidence and Provide contact details.

The actors make the audience explicit. The User (Motorist) is the primary human actor. The Registered SV Account Holder is drawn as a generalisation of the User — that is, a registered holder is a special kind of motorist who can do everything an ordinary motorist can, plus account-specific actions such as saving favourites. Two system actors support the motorist: Location Services, a «GPS Provider», and Mapping Service, a «Routing Provider». The remaining lanes, included for context, show the Fuel Retailer submitting and capping prices, a time-triggered Scheduler flagging stale prices and rolling over the daily cap, the Service Victoria Admin approving retailers and reviewing issues, and Consumer Affairs Victoria monitoring compliance and taking enforcement action.

4.4 Relationships and logic

The choice of relationship in each case is deliberate. Detecting location, retrieving prices and displaying results are modelled as «include» because a search is meaningless without them — they are mandatory sub-steps. Filtering, sorting and viewing the price cap are modelled as «extend» because the search is complete and useful without them; they are optional enhancements the motorist may or may not invoke. The Registered SV Account Holder is modelled with a generalisation rather than a separate actor because it shares all of the motorist's behaviour and merely adds to it; duplicating the shared use cases for a second actor would be both inaccurate and cluttered. This careful separation of mandatory, optional and specialised behaviour is what the assessment rewards, and it sets up the improvement, where new alert services attach to the registered-holder specialisation.

4.5 Business meaning

This single page tells an executive what Servo Saver does and for whom: it is, at its heart, a search-and-compare service for motorists, wrapped in the governance and data-supply

machinery needed to keep the prices accurate and enforceable. It also reveals an absence that motivates the improvement — there is no service by which the system reaches out to the motorist; every motorist use case begins with the motorist acting first.

5. Detailed (Expanded) Use Case: Search Nearby Fuel Stations

5.1 What this section adds and why

The overall diagram names the services but does not explain how any one of them works in detail. To make a single service concrete enough that a developer could build it, the analyst "expands" one use case into an expanded (fully-dressed) use case description with three sections: the identification and description, the flow of activities, and the exception conditions. The use case chosen here is UC1 — Search nearby fuel stations, the system's central service. The expanded description is presented in Table 5.1, and the two actors it involves are the User (Motorist) as the primary actor and Location Services as the supporting «GPS Provider».

Use Case Name:	Search nearby fuel stations
Scenario:	A motorist searches for the cheapest nearby fuel using the Servo Saver feature of the Service Victoria app.
Triggering Event:	Motorist wants to find cheap fuel nearby and opens the "Servo Saver" tile.
Brief Description:	When the motorist opens Servo Saver, the system determines the user's location (via GPS or manual entry), retrieves real-time fuel prices and the locked daily price caps, and displays nearby stations on a map so the motorist can compare prices and select a station.
Actors:	User (Motorist) — primary; Location Services «GPS Provider» — secondary (system actor).
Related Use Cases:	Includes: Detect user location, Retrieve real-time fuel prices, Display fuel price results. Extended by: Enter location manually, Filter (type / brand), Sort (price / distance), View daily price cap (tomorrow).
Stakeholders:	Fuel Retailer: expects submitted prices to be displayed accurately. Service Victoria Admin: to maintain service quality and data accuracy. Consumer Affairs Victoria: to ensure fuel price transparency and retailer compliance. Mapping Service provider: supplies map tiles and routing for downstream use cases.
Preconditions:	Service Victoria app is installed and the Servo Saver feature is available. Fuel price database is online with current retailer-submitted prices and locked daily caps. The device has an active network connection.
Postconditions:	Nearby stations with current prices are displayed on the map and a station can be selected.

	The search request is logged for analytics. Displayed prices reflect the latest validated retailer submissions.	
Flow of Activities:	Actor	System
	1. Motorist opens the Service Victoria app and taps the “Servo Saver” tile.	1.1 Load the Servo Saver feature. 1.2 Check GPS permission status.
	2. Motorist confirms location access.	2.1 Request coordinates from Location Services. 2.2 Receive Coordinates [captured]. 2.3 Query fuel price database and retrieve locked daily price caps (in parallel). 2.4 Render map of nearby stations with prices.
	3. Motorist reviews prices on the map and applies filter / sort if desired.	3.1 Re-display results per filter / sort selection.
	4. Motorist selects a fuel station.	4.1 Display station details (prices, address, opening hours).
Exception Conditions:	1.2 If GPS permission is not granted, the system shows the manual-entry prompt; the motorist enters a suburb / postcode, the system geocodes it, and the flow resumes at step 2.3. 2.1 If Location Services cannot return coordinates, the system falls back to manual entry (as in 1.2). 2.3 If the fuel price database is unreachable, the system displays a service-unavailable message and offers a retry. 2.4 If no stations are found within the radius, the system shows an empty state and the motorist can re-enter the location or widen the search. 4.1 If station details fail to load, the system shows cached details with a “last updated” warning.	

Table 5.1. Expanded (fully-dressed) use case description for UC1 — Search nearby fuel stations: identification, actors, related use cases, stakeholders, pre/postconditions, the actor-and-system flow of activities, and the numbered exception conditions.

5.2 Section one — identification, description and stakeholders

The expanded description records the essential facts about the use case. Its scenario is that a motorist searches for the cheapest nearby fuel using Servo Saver, triggered when the motorist opens the Servo Saver tile. The brief description states that the system determines the user's location, retrieves real-time prices and the locked daily caps, and displays nearby stations on a map for comparison and selection. The primary actor is the User (Motorist); the secondary system actor is Location Services. The description lists the included use cases (detect location, retrieve prices, display results) and the extending ones (manual entry, filter, sort, view cap), and it names the stakeholders and their interests — the fuel retailer who expects accurate display of submitted prices, the Service Victoria Admin who maintains data quality, Consumer Affairs Victoria which oversees transparency and compliance, and the mapping-service provider that supplies tiles and routing.

The preconditions and postconditions bound the use case. Before it can run, the Service Victoria app must be installed and Servo Saver available, the fuel-price database must be online with current submissions and locked caps, and the device must have network access. After it completes successfully, nearby stations with current prices are displayed and a station can be selected, the search is logged for analytics, and the prices shown reflect the latest validated retailer submissions.

5.3 Section two — the flow of activities

The flow is written as a numbered dialogue in which each actor action is paired with the system's response, which is exactly how a programmer would later turn it into code. In step 1 the motorist opens the app and taps the Servo Saver tile, and the system responds by loading the feature (1.1) and checking GPS permission (1.2). In step 2 the motorist confirms location access; the system requests coordinates from Location Services (2.1), receives the captured coordinates (2.2), queries the price database and retrieves the locked daily caps in parallel (2.3), and renders the map of nearby stations with prices (2.4). In step 3 the motorist reviews the prices and may apply a filter or sort, and the system re-displays the results accordingly (3.1). In step 4 the motorist selects a station and the system displays its details — prices, address and opening hours (4.1).

5.4 Section three — exception conditions

The exceptions are tied to the step numbers so that each failure has a defined response rather than being left to chance. At 1.2, if GPS permission is not granted, the system shows the manual-entry prompt, the motorist enters a suburb or postcode, the system geocodes it, and the flow resumes at 2.3. At 2.1, if Location Services cannot return coordinates, the system falls back to that same manual entry. At 2.3, if the price database is unreachable, the system shows a service-unavailable message and offers a retry. At 2.4, if no stations are found within the radius, the system shows an empty state and lets the motorist re-enter the location or widen the search. At 4.1, if station details fail to load, the system shows cached details with a "last updated" warning. Each exception names the triggering step, the cause and the recovery, so the behaviour is fully specified.

5.5 Business meaning

The expanded use case demonstrates that the central service has been thought through to the level of individual system responses and failure handling. For the business, this is the evidence that the analysis is implementable rather than aspirational; for this report, it also establishes the

pattern that the new Set price drop alert use case follows later, including its reuse of the same location-detection step and the same GPS fallback.

6. Class Diagram

6.1 What this diagram is and why it exists

A class diagram describes the system's structure rather than its behaviour. It shows the kinds of "things" (classes) the system stores and works with, the data each holds (attributes) and the operations each can perform (methods), and how the classes are connected. Each class is drawn as a box in three parts: the name at the top, the attributes in the middle (each with a type, such as String or Float), and the methods at the bottom (shown with brackets). It is the blueprint from which a database and the program's objects are built.

6.2 How to read the notation

Four kinds of connection appear in this diagram. A plain line is an association — a general relationship, usually labelled with the roles and with multiplicities such as 1, 0..* (zero or many) or 1..* (one or many) that say how many of each class take part; note that a bare multiplicity of 1 means exactly one (1..1), not zero-or-one. A hollow diamond is an aggregation — a "has-a" relationship in which the part can exist independently of the whole. A filled diamond is a composition — a stronger "owns-a" relationship in which the part cannot exist without the whole. A hollow triangle is a generalisation — an "is-a" inheritance relationship. The word {abstract} on a class means it is a general template that is never created directly, only through its more specific subclasses.

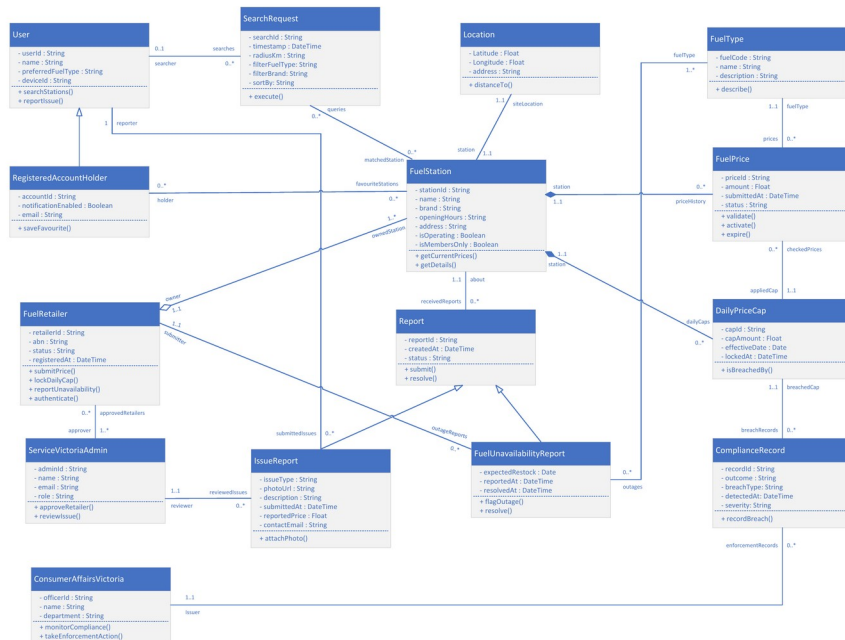


Figure 6.1. Class diagram for the current Servo Saver system, showing the user and actor classes, the search and location classes, and the core FuelStation, FuelPrice, FuelType and DailyPriceCap classes with their associations, compositions and the abstract Report hierarchy.

6.3 Element-by-element walk-through

At the top of the people side of the model is the User class, holding `userId` and `deviceId` and offering `searchStations()` and `reportIssue()`. `RegisteredAccountHolder` generalises User — inheriting its behaviour and adding `accountId`, a list of favourites and `saveFavourite()`. `FuelRetailer` holds `retailerId`, `abn` and `status` and offers `submitPrice()`, `lockDailyCap()` and `reportUnavailability()`; `ServiceVictoriaAdmin` offers `approveRetailer()` and `reviewIssue()`; and `ConsumerAffairsVictoria` offers `monitorCompliance()` and `takeEnforcementAction()`. These classes mirror the actors seen in the use case diagram, which is a healthy sign that the structural and behavioural models agree.

On the data side, a `SearchRequest` records a `searchId`, `timestamp` and `radiusKm` and can `execute()`; it is performed by one User (the searcher) and queries many `FuelStation` objects. A `Location` class stores `latitude`, `longitude` and `address` and can compute `distanceTo()`. The heart of the model is `FuelStation`, which holds `stationId`, `name`, `brand` and `openingHours` and offers `getCurrentPrices()` and `getDetails()`. A `FuelStation` has exactly one `Location`, is owned by one

FuelRetailer, and is composed of its FuelPrice records and its DailyPriceCap records. FuelPrice holds priceId, amount, submittedAt and status and offers validate(), activate() and expire(); FuelType holds fuelCode and name; and DailyPriceCap holds the locked maximum price that a FuelPrice is validated against. Finally, an abstract Report class (with reportId, createdAt and status) generalises the specific report types raised about stations.

6.4 Relationships and logic

The relationship choices carry real meaning. FuelPrice and DailyPriceCap are joined to FuelStation by composition (the filled diamond) because a price or a cap has no meaning without the station it belongs to — delete the station and its prices should vanish with it. The link between FuelStation and FuelRetailer is an aggregation because a station belongs to a retailer but is a meaningful entity in its own right. The User-to-RegisteredAccountHolder link is a generalisation, matching the actor generalisation in the use case diagram. Multiplicities encode the business rules: a station has zero-to-many prices over time (its price history) but, as the CRC cards note, only one active price per fuel type at a time; a retailer owns one-to-many stations; and a search matches zero-to-many stations.

6.5 The CRC cards

Class-Responsibility-Collaborator (CRC) cards are a complementary, plain-language way of describing the most important classes. Each card lists what a class is responsible for knowing and doing, and which other classes it must collaborate with to do its job. Tables 6.1 and 6.2 present the cards for the two classes at the centre of the system, FuelStation and FuelPrice.

Class: FuelStation	
Superclass: none Sub-class: none	
Responsibilities	Collaborators
Know station identity, brand and opening hours	—
Provide current prices for search results	FuelPrice, FuelType, SearchRequest
Maintain price history and locked daily price caps	FuelPrice, DailyPriceCap
Provide station details and position for map / directions	Location
Receive and hold reports raised about the station	Report (IssueReport, FuelUnavailabilityReport)
Know its owning retailer	FuelRetailer
Attributes • stationId : String • name : String	

• brand : String • openingHours : String
Relationships Generalisation: none Association: Location (1 to 1); SearchRequest (0..* to 0..*); Report (1 to 0..*) Aggregation: part of FuelRetailer (owner 1 ◇ 1..* stations) Composition: whole of FuelPrice (1 ◆ 0..*); whole of DailyPriceCap (1 ◆ 0..*)
Business Rules • A station can only be registered by a FuelRetailer that has been approved by ServiceVictoriaAdmin. • A station holds at most one active FuelPrice per FuelType at any time. • A station appears in search results only when it lies within the search radius of the request.

Table 6.1. CRC card for FuelStation: its responsibilities, collaborators, attributes, relationships and business rules — including that a station holds at most one active price per fuel type at a time.

Class: FuelPrice	
Superclass: none Sub-class: none	
Responsibilities	Collaborators
Know its amount, submission time and lifecycle status	—
Validate the submitted amount against the locked daily cap	DailyPriceCap
Activate a validated price so it is shown to motorists	FuelStation
Expire / supersede itself when a newer price is activated or it goes stale	FuelStation, Scheduler (flag stale)
Know which fuel type it prices	FuelType
Supply price data to search results	SearchRequest
Attributes • priceId : String • amount : Float • submittedAt : DateTime • status : String (submitted / validating / active / rejected / stale)	
Relationships Generalisation: none Association: FuelType (0..* prices to 1 fuelType); DailyPriceCap (0..* checkedPrices to 1 appliedCap) Aggregation: none Composition: part of FuelStation (station 1 ◆ 0..* priceHistory) — cannot exist without its station	
Business Rules • A submitted amount must not exceed the locked DailyPriceCap for that fuel type and effective date. • Only a validated price can become active; activating a new price supersedes the previous active price for that fuel type. • A price not refreshed by the retailer is flagged stale by the Scheduler and removed from display.	

Table 6.2. CRC card for FuelPrice: its responsibilities (validate against the cap, activate, expire), collaborators, attributes and the lifecycle that the state machine in Section 7 models in full.

The FuelStation card records that the station knows its identity, brand and opening hours, provides current prices for search results (collaborating with FuelPrice, FuelType and SearchRequest), maintains its price history and locked caps (collaborating with FuelPrice and DailyPriceCap), supplies its position for maps and directions (collaborating with Location), and holds reports raised about it. Its business rules note that a station can only be registered by an approved retailer and that it appears in results only when within the search radius. The FuelPrice card records that a price knows its amount, submission time and lifecycle status, validates itself against the locked daily cap, activates so motorists can see it, and expires or is superseded when a newer price arrives or it goes stale. That lifecycle — submitted, validating, active, rejected, stale — is exactly what the state-machine diagram in the next section models in full.

7. State Machine Diagram: the FuelPrice Object

7.1 What this diagram is and why it exists

A state-machine diagram tracks the complete life of a single object — here, one FuelPrice taken from the class diagram — as it moves through the different states it can occupy, and the events that move it from one state to the next. It is important to be precise on one point that is often misunderstood: a state machine is always about one object. There is no such thing as an "overall" state machine; "overall" belongs to the use case diagram. This diagram therefore follows a single fuel price from the moment a retailer submits it to the moment it is destroyed.

7.2 How to read the notation

A rounded rectangle is a state, and inside it the entry, do and exit lines describe what happens when the object enters the state, while it remains there, and as it leaves. An arrow between states is a transition, labelled in the form event [guard] / action — the event that triggers it, an optional condition in brackets that must be true, and an action performed as it fires. A solid dot is the start; a dot inside a ring is a final state. A diamond is a choice, where the path taken depends on guards evaluated at that moment. A solid bar is a fork or join for concurrent behaviour. A larger dashed rounded rectangle is a composite state, which contains its own smaller states; a circled H inside one is a history marker that remembers which inner state was active; and a circled cross is a terminate, meaning the object is destroyed outright.

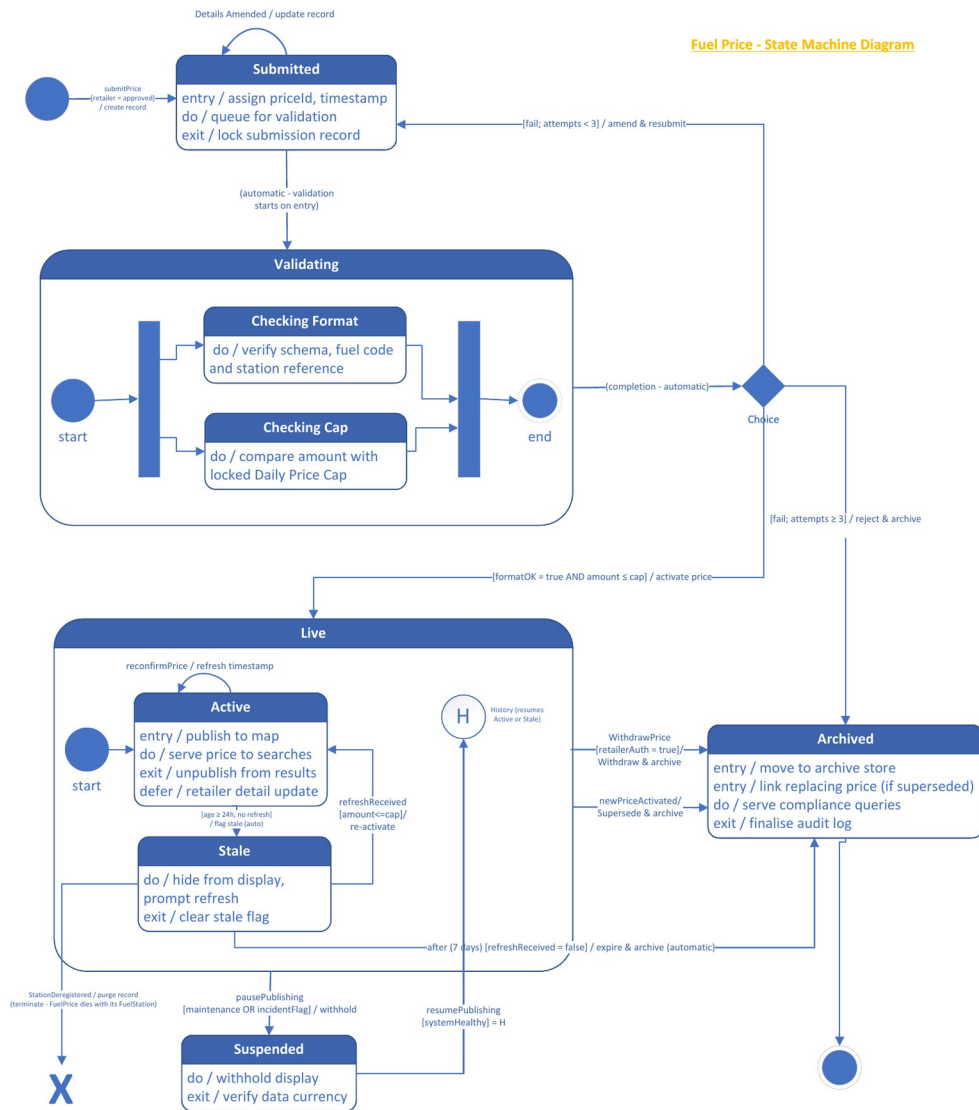


Figure 7.1. State machine for a single `FuelPrice` object, with nine states. The `Validating` composite runs two checks concurrently (fork/join) and has its own start and end; the `Live` composite holds the `Active` and `Stale` substates with a history marker; a choice routes validated prices; and a `terminate` destroys the price when its station is deregistered.

7.3 Element-by-element walk-through

A FuelPrice begins at the start marker and enters Submitted when a retailer submits a price, where it is assigned an identifier and timestamp and queued for validation. From Submitted it moves automatically into Validating, a composite state that does two things at once. Inside Validating, a start marker leads to a fork that launches two concurrent checks: CheckingFormat verifies the data schema, fuel code and station reference, while CheckingCap compares the submitted amount against the locked DailyPriceCap. A join waits for both checks to finish and leads to the composite's own end (its region-final marker). This is why the diagram says Validating has both its own start and its own end: it is a self-contained piece of work that completes and then hands control back.

On completion the price reaches a choice. If the data failed or the amount exceeded the cap, one guarded path applies — if fewer than three attempts have been made the price returns to Submitted for correction and resubmission, otherwise it is rejected and archived. If the checks passed, the other guarded path activates the price and it enters Live. Live is the second composite state, of the "exactly one substate active" kind: inside it the price is normally Active — published to the map and served to searches — and can reconfirm itself to refresh its timestamp. If it ages past twenty-four hours with no refresh it moves to Stale, where it is hidden and the retailer is prompted to refresh; a refresh returns it to Active. A history marker inside Live remembers whether the price was Active or Stale so that, after an interruption, it can resume the correct substate.

Three events take the price out of Live. If a newer price is activated, this one is superseded and archived; if the retailer withdraws it, it is withdrawn and archived; and if it sits stale for seven days, it expires and is archived. The price can also be temporarily suspended (for maintenance or an incident) and later resumed, which is the situation the history marker exists to handle. Once Archived, the price is retained for compliance and, after the seven-year retention period, reaches the final state and is destroyed. Separately, a terminate marker shows that if the station itself is deregistered, the price is purged immediately — a direct consequence of the composition relationship in the class diagram, where a price cannot exist without its station.

7.4 Relationships and logic

The diagram is deliberately limited to nine states so that it stays readable while still exercising every important state-machine feature: two composite states, concurrent substates with a fork and join, a choice with mutually exclusive guards, a history marker, a terminate and a final state. The composite start/end question is handled correctly and differently in the two composites. Validating has both a start and an end because it genuinely completes on its own and then passes

control to the choice. Live has a start but no internal end, because a live price is never "finished" from the inside — it is always taken out by an external event such as being superseded, withdrawn or expiring. Adding an end to Live would therefore be wrong, and the diagram is annotated to make that design decision explicit.

7.5 Business meaning

For the business, this diagram is the guarantee that a published price is trustworthy: every price is validated against the legal daily cap before it goes live, it is automatically marked stale and then expired if it is not kept current, and it is retained for the compliance period before being destroyed. It also pinpoints the data the improvement depends upon — the moment a price becomes Active is precisely the event that the proposed Price Drop Alerts feature listens for when deciding whether to notify a motorist.

8. User Interface Design: the Good, the Bad and the Ugly

This section evaluates the Servo Saver interface as a motorist actually experiences it, based on a hands-on walk-through of the live application. The screens captured during that walk-through are shown in Figures 8.1 to 8.4 and referenced throughout. To respect the assessment's guidance on not occupying space with large pictures, the captures are reproduced at a compact size and the full-resolution versions are placed in Appendix A.

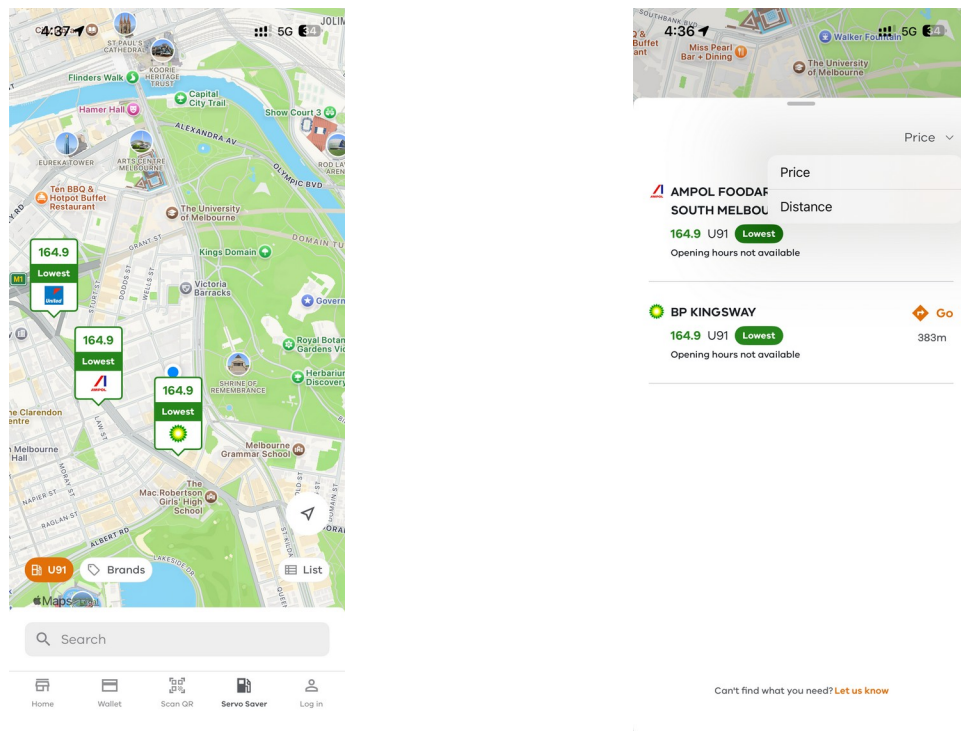


Figure 8.1 — Map view

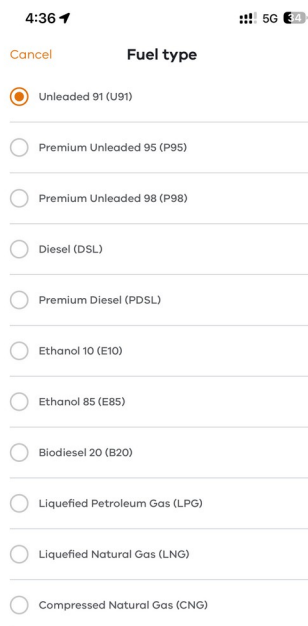


Figure 8.3 — Fuel-type selector

Figure 8.2 — List view

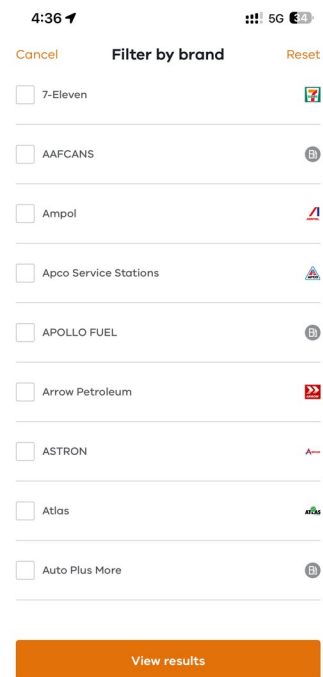


Figure 8.4 — Brand filter

Figure 8.1–8.4. The current Servo Saver interface, captured from the live application: (8.1) the map view, (8.2) the ranked list with the “Opening hours not available” line, (8.3) the fuel-type selector, and (8.4) the brand filter with inconsistent logos.

8.1 The good

The strongest design decision is the map-first price view in Figure 8.1. Prices appear as colour-coded pins on a familiar map, and the cheapest carry a green “Lowest” badge, so a motorist can answer “where is the cheapest fuel near me?” at a glance, without reading a single number. Anchoring the data to a real map, with the user’s own location shown, ties the prices to genuine wayfinding rather than an abstract list. A second strength, visible on the station-detail screen, is the “Tomorrow’s price cap” panel: showing today’s price beside tomorrow’s locked cap is a genuinely forward-looking feature that lets a motorist decide whether to fill up now or wait. The comprehensive fuel-type and brand filters (Figures 8.3 and 8.4) and the map/list toggle with a price or distance sort (Figure 8.2) give the user flexible, well-considered ways to narrow the results, and the whole feature inherits the trust and single sign-in of the Service Victoria platform.

8.2 The bad

The most significant weakness is that the experience is entirely pull-based: the motorist must remember to open the app and look. Fuel prices change daily, yet there is no way to be told when a price falls, so the benefit depends on the user checking at the right moment. Closely related,

there is no obvious way to save a station or build a watchlist, so regular users must re-search every time. Trust in the figures is weakened by a single, app-wide "updated" timestamp rather than a per-station freshness indicator, which means a motorist cannot easily judge how current any one price is. Smaller frictions include sort options limited to price or distance independently rather than a combined "cheapest within a set distance", and a search field that gives no guidance on whether it expects a suburb, a station or a brand.

8.3 The ugly

Two details actively undermine credibility. The first is the "Opening hours not available" line, which appears beneath the prices on most station listings (visible in Figure 8.2 and on the detail screen). Presented as plain text where useful information should be, it reads as broken rather than merely absent, and it sits exactly where the motorist is judging whether to trust the listing. The second is the brand filter in Figure 8.4, where some retailers show real logos while many fall back to an identical grey placeholder icon; the inconsistency makes an otherwise polished feature look unfinished. Neither problem is about the price data itself — they are presentation defects — but because they appear at the point of decision they do disproportionate harm to user confidence, and both are straightforward to fix.

9. Proposed Improvement: Price Drop Alerts

9.1 What the improvement is

The proposed improvement is a Price Drop Alerts feature. A registered Service Victoria account holder chooses a fuel type and a target price, sets a search radius around their location and optional quiet hours, and the system then watches newly published prices and sends a notification when a station within range falls to or below the target. In effect, the feature turns Servo Saver from a tool the motorist must remember to consult into an assistant that reaches out at the moment it can save the user money. Figure 9.1 shows how the existing station-detail screen would carry the new "Set price alert" control alongside several supporting interface fixes.

Servo Saver — Improved Station Detail (proposed)

"After" mockup to pair with the live screenshot (Figure 5). Coral callouts mark the new / improved elements.

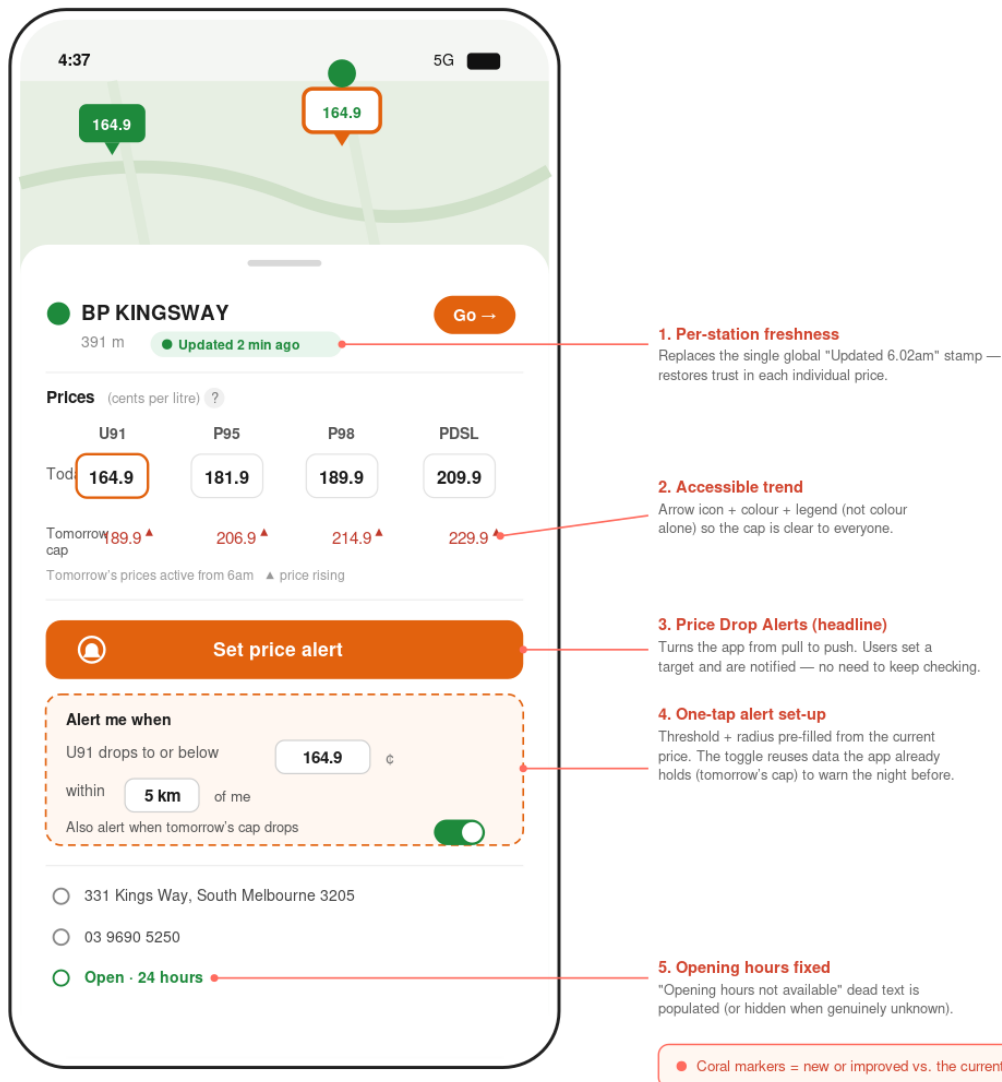


Figure 9.1. Proposed improved station-detail screen. Coral callouts mark the new and improved elements: per-station freshness, an accessible price-trend indicator, the headline "Set price alert" action, a one-tap alert set-up that reuses tomorrow's price-cap data, and a fix for the "opening hours" defect.

9.2 How it integrates with the existing system

The feature is deliberately designed to reuse what the system already has rather than to bolt on a parallel system. It detects the monitoring location through the same Detect user location service that UC1 already uses, including the same fall-back to manual suburb entry when GPS is unavailable. It validates a user's target against the same locked DailyPriceCap that the state machine already applies when validating a retailer's submission. And it listens for the very event the state machine already raises — a FuelPrice becoming Active — to decide whether any saved

alert is now satisfied. Because all of these are existing components, the improvement adds value with minimal new infrastructure, which is itself part of the business argument for it.

9.3 Why it adds value

The value case rests on engagement, retention and tangible savings. Evidence from mobile-engagement research is strong: an Airship (2024) benchmark study found that users who received push notifications in their first ninety days were retained at close to three times the rate of those who received none, and that well-targeted notifications can lift app engagement substantially. For a government platform whose goal is to have citizens actually use its services, a feature that demonstrably increases return visits is valuable in its own right. The savings case is equally concrete: because the ACCC (2024) estimates that buying at the low point of the price cycle is worth around \$333 a year to a Melbourne motorist, an alert that tells the user when that low point has arrived converts a theoretical saving into a realised one. This directly amplifies the policy intent behind Victoria's Fair Fuel Plan (Premier of Victoria, 2025), which is to help motorists pay less and to reward competitively priced retailers. Finally, because the feature reuses the existing daily-price-cap data (Premier of Victoria, 2025), it can warn users the evening before a price change rather than after it — a level of foresight competitors cannot easily match, strengthening the platform's reputation and the goodwill of the citizens it serves.

Value driver	How Price Drop Alerts delivers it	Evidence
Engagement & retention	Opt-in push alerts bring users back to the platform at the moment value is highest	Airship (2024)
Motorist savings	Tells the user when the cycle low or a lower locked cap has arrived	ACCC (2024)
Policy alignment	Steers demand toward competitively priced stations, amplifying the Fair Fuel Plan	Premier of Victoria (2025)
Low build cost and risk	Reuses existing location, price, daily-cap and account components	This report (Sections 7, 10)

Table 9.1. The value drivers of Price Drop Alerts and the evidence supporting each.

9.4 Weighing the trade-offs

A responsible proposal must also weigh the perspectives that pull against it. The most obvious risk is notification fatigue: research on mobile engagement shows that poorly targeted or too-frequent notifications can prompt users to disable them or uninstall the app altogether (Airship, 2024), which would defeat the purpose. The design answers this directly — alerts are opt-in, are bounded by a user-chosen threshold and radius, respect quiet hours, and are limited in number per account — so the user, not the system, controls the volume. A second perspective is that of

fuel retailers, who might view alerts as intensifying price competition; on balance, however, the feature rewards the retailers who price most competitively by steering demand toward them, which is consistent with the policy intent rather than opposed to it. A third consideration is privacy: storing a user's monitoring location and preferences introduces a duty of care, which is why the Limitations section flags a security review as a precondition of implementation. Acknowledging and resolving these divergent views, rather than ignoring them, is what makes the case for the feature credible.

9.5 Supporting interface improvements

Price Drop Alerts is the headline feature, but it is most effective when paired with the smaller interface fixes identified in Section 8, which Figure 9.1 also illustrates. A per-station freshness indicator replaces the single app-wide timestamp, so a motorist can see at a glance how current each individual price is and can trust the figure they are about to act on. An accessible price-trend indicator pairs the existing colour cue with an explicit up or down icon and a short legend, so the meaning of the price cap is clear to every user, including those who cannot distinguish the colours. The "opening hours not available" defect is resolved by populating the field where the data exists and suppressing the row where it genuinely does not, removing the broken-looking text from the point of decision. Finally, a favourites or watchlist capability gives regular users a way to save the stations they care about, and those saved stations become natural candidates for alerts — so the small fixes and the headline feature reinforce one another.

9.6 Indicative impact

The combined effect can be illustrated conservatively. The ACCC (2024) figure of roughly \$333 a year in potential savings represents the gap between buying at the top and the bottom of the price cycle; an alert does not capture all of that gap, but even helping a motorist capture a modest share of it turns an abstract figure into real money saved, repeated across a large user base. On the engagement side, the Airship (2024) benchmark that opted-in, notified users are retained at close to three times the rate of non-notified users implies that an alerts feature would materially lift the proportion of citizens who keep using the Service Victoria platform over time. For a government service, that improved retention is not a commercial end in itself but a measure of how effectively the platform is fulfilling its public purpose. Crucially, because the feature is additive and reuses existing data and components, these benefits are achieved without the cost and risk of building new price-collection or location infrastructure — which is what makes the return on the investment attractive.

10. Incorporating the Improvement into the Models

The assessment requires the improvement to be shown in at least two diagrams, with new use cases expanded into a detailed use case. This report exceeds that requirement: it shows the improvement in an improved overall use case diagram, an expanded description of the new use case, and an improved class diagram with new CRC cards. Throughout, the new and changed elements are highlighted in coral so that a reader can immediately see what the improvement adds to the legacy system.

10.1 Improved overall use case diagram

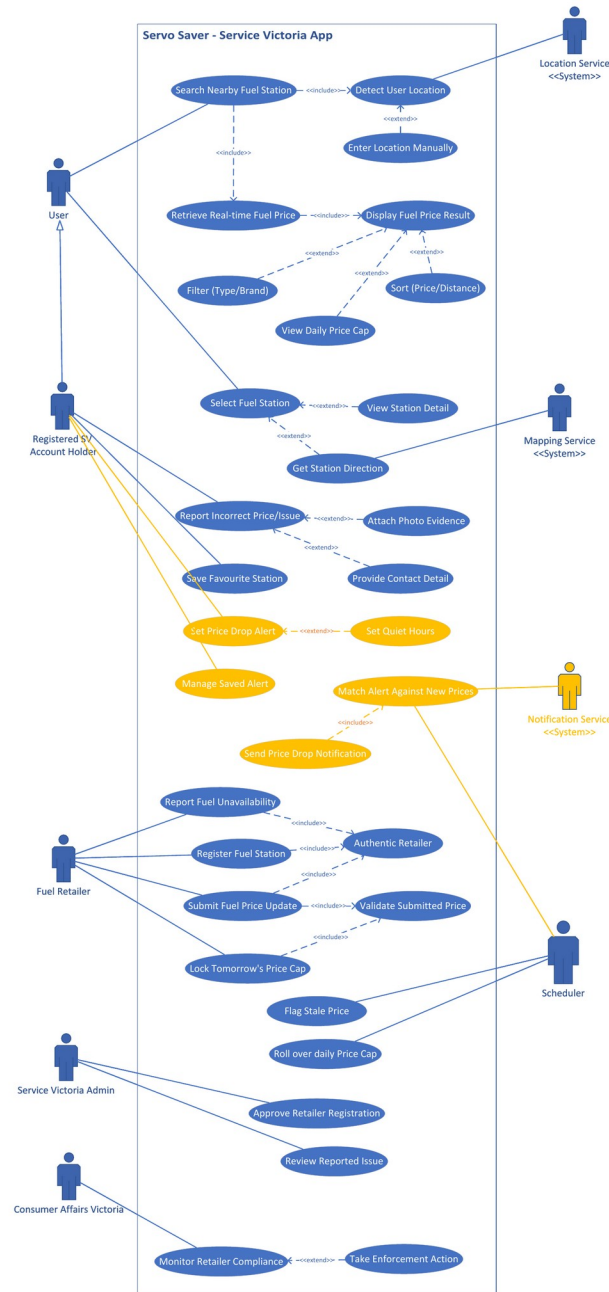


Figure 10.1. Improved overall use case diagram. The new price-drop-alert services and the new Notification Service «Push Provider» actor are highlighted in coral and attach to the Registered SV Account Holder specialisation and the existing Scheduler.

Figure 10.1 adds a new group of services, the price-drop-alert flows, to the diagram from Section 4. The new use cases are Set price drop alert, Set quiet hours, Manage saved alerts, Match alerts against new prices and Send price drop notification. Set price drop alert is the service the user invokes to create an alert; it is the one expanded in detail below. Set quiet hours «extends» it, because choosing not to be disturbed at night is an optional addition to creating an alert. Manage saved alerts lets the holder pause, edit or delete existing alerts. Send price drop notification is the system's outgoing service; it «includes» Match alerts against new prices, because sending a notification always requires first checking saved alerts against newly activated prices.

Two integration points connect the new services to the legacy model. First, the new use cases attach to the Registered SV Account Holder — the specialised motorist actor that already existed in Figure 4.1 — because alerts are a benefit of holding an account; an anonymous motorist cannot set one. This is why the earlier generalisation matters: the improvement extends the specialised actor without disturbing the ordinary motorist's services. Second, a new system actor, the Notification Service «Push Provider», is introduced to deliver the messages, and the existing time-triggered Scheduler is reused to run the matching. The diagram therefore demonstrates that the improvement is genuinely woven into the existing system rather than standing apart from it.

10.2 Expanded new use case: Set price drop alert

Use Case Name:	Set price drop alert (new use case — Price Drop Alerts improvement)
Scenario:	A registered account holder creates an alert to be notified when fuel near them drops below a target price.
Triggering Event:	Account holder wants to be told when fuel gets cheap and taps “Set price drop alert”.
Brief Description:	The holder defines fuel type, threshold price, search radius and optional quiet hours. The system determines the monitoring location (via GPS or manual entry), validates the criteria, and saves the alert so the Scheduler can match it against newly activated fuel prices.
Actors:	Registered SV Account Holder — primary; Location Services «GPS Provider» — secondary. (Scheduler and Notification Service «Push Provider» act in the related “Send price drop notification”.)
Related Use Cases:	Includes: Detect user location (reused from the legacy system — integration point). Extended by: Set quiet hours. Related new: Manage saved alerts; Send price drop notification «includes» Match alerts against new prices.
Stakeholders:	Fuel Retailer: alerts steer customers to the lowest-priced stations, rewarding competitive pricing. Service Victoria Admin: higher engagement and return visits to the SV app.

	Notification Service provider: delivers the push messages within agreed service levels. Consumer Affairs Victoria: alerts increase price transparency for motorists.	
Preconditions:	User holds a registered SV account and is signed in. Fuel price database is online. The Servo Saver feature and alert criteria screen are available.	
Postconditions:	A PriceAlert record (status = active) is stored and linked to the account holder. The alert is registered with the Scheduler for matching against newly activated prices.	
Flow of Activities:	Actor	System
	1. Holder taps "Set price drop alert".	1.1 Verify the holder is signed in. 1.2 Display the alert criteria form.
	2. Holder selects fuel type and threshold price.	2.1 Validate threshold against a plausible range.
	3. Holder sets the search radius (or accepts the default).	3.1 Determine monitoring location via Detect user location (GPS or manual suburb).
	4. Holder optionally sets quiet hours (Set quiet hours extension).	4.1 Attach quietHours to the alert.
	5. Holder saves the alert.	5.1 Create PriceAlert (status = active) linked to the account. 5.2 Confirm creation; show in saved alerts. 5.3 Register the alert with the Scheduler.
Exception Conditions:	1.1 If the user is not signed in (guest motorist), the system prompts registration / sign-in; the flow resumes at step 1 once authenticated (alerts are exclusive to Registered SV Account Holders). 2.1 If the threshold is implausible (≤ 0 , or above the locked DailyPriceCap for that fuel), the system rejects it and asks for a corrected value. 3.1 If GPS is unavailable or permission is denied, the system falls back to manual suburb / postcode entry (same fallback as UC1, exception 1.2). 5.1 If an identical alert already exists (same fuel, radius and threshold), the system blocks the duplicate and offers to edit the existing alert instead. 5.3 If the active-alert limit (10) is reached, the system asks the holder to delete or pause an alert via Manage saved alerts before saving the new one.	

Table 10.1. Expanded description of the new Set price drop alert use case: identification, actors, related use cases (including the reused «include» of Detect user location), stakeholders, pre/postconditions, the flow in which the system creates a PriceAlert and registers it with the Scheduler, and the five exception conditions.

So that a programmer could build the new feature, the Set price drop alert use case is expanded into the same three-section format used for UC1. In identification, the scenario is that a registered account holder creates an alert to be notified when fuel near them drops below a target price, triggered when the holder taps "Set price drop alert". The primary actor is the Registered SV Account Holder and the secondary actor is Location Services. Critically, the description records

that this use case «includes» Detect user location — explicitly named as the reused legacy integration point — and is «extended by» Set quiet hours, with Manage saved alerts and Send price drop notification as related new use cases. The stakeholders and their interests are spelled out: fuel retailers, whose competitive pricing is rewarded when alerts steer customers to the cheapest stations; the Service Victoria Admin, who gains higher engagement and return visits; the notification-service provider, who must deliver messages within agreed service levels; and Consumer Affairs Victoria, whose transparency objective is advanced.

The flow of activities pairs each holder action with a system response. The holder taps Set price drop alert; the system verifies they are signed in and displays the criteria form. The holder selects a fuel type and threshold price; the system validates the threshold against a plausible range. The holder sets a search radius; the system determines the monitoring location via the reused Detect user location step. The holder optionally sets quiet hours; the system attaches them to the alert. Finally the holder saves the alert, and the system creates a PriceAlert record with status active linked to the account, confirms it in the saved-alerts list, and registers it with the Scheduler so it can be matched against newly activated prices. The postconditions confirm exactly this: a PriceAlert (status = active) is stored and linked to the holder, and registered with the Scheduler.

Five exception conditions, each tied to a step, make the behaviour robust. At 1.1, a guest motorist who is not signed in is prompted to register or sign in, and the flow resumes once authenticated — reinforcing that alerts are exclusive to registered holders. At 2.1, a threshold that is implausible (zero or below, or above the locked DailyPriceCap for that fuel) is rejected and a corrected value requested — a direct reuse of the legacy cap. At 3.1, if GPS is unavailable the system falls back to manual suburb entry, the very same fallback as UC1's exception. At 5.1, an identical existing alert is blocked and the holder is offered the chance to edit the existing one. At 5.3, if the ten-alert limit is reached, the holder is asked to delete or pause an alert via Manage saved alerts before saving. Each exception names its trigger and its resolution, so the new feature is specified to the same depth as the legacy use case.

is matched against many triggering prices, and a price may match many alerts). It also has a composition relationship to `AlertNotification` — the notifications it raises cannot exist without it. Its business rules tie the improvement back to the legacy model: only a registered holder can create an alert, an alert fires only when an active `FuelPrice` for its fuel code, within its radius, falls to or below its threshold, and no notification is sent during quiet hours — deferred matches are sent once quiet hours end.

10.3.2 `AlertNotification` (new class)

`AlertNotification` represents a single message sent to the holder when an alert is satisfied. Its attributes are `notificationId`, `channel` (push or email), `sentAt` and `deliveryStatus` (pending, delivered or failed). Its responsibilities are to know its channel, send time and delivery status, to compose the alert message (the station, fuel, new price and the saving), to deliver that message through the chosen channel by collaborating with the Notification Service push provider, to confirm delivery or retry on failure, and to record the triggering price so the holder can verify the saving. It reaches the `FuelPrice` and the holder through its owning `PriceAlert` rather than holding its own direct links, which keeps the design clean. Modelling notifications as a separate class, owned by the alert through composition, means the system keeps a full, auditable history of what was sent and when, without cluttering the `PriceAlert` class itself.

10.3.3 New relationships to the legacy classes

The new classes plug into the existing model at two well-chosen points. `PriceAlert` associates with the existing `RegisteredAccountHolder` (shown as 0..* `savedAlerts` to 1 holder), so alerts hang off the same specialised actor the use case diagram uses, and an account holder can keep many alerts. `PriceAlert` also associates with the existing `FuelPrice` (0..* `matchedAlerts` to 0..* `triggeringPrices`), which is how an alert is tested against the prices the state machine activates. No existing class is altered or removed; the improvement is purely additive. This is the structural evidence that the new feature integrates with the legacy system rather than duplicating it — it reuses `RegisteredAccountHolder` for ownership, `FuelPrice` and `FuelStation` for matching, `Location` for the radius test, and `DailyPriceCap` for threshold validation.

10.4 CRC cards for the new classes

Class: <code>PriceAlert</code> (new class)	
Superclass: none	Sub-class: none
Responsibilities	Collaborators
Know fuel type, threshold price, radius and quiet	—

hours	
Belong to the account holder who created it	RegisteredAccountHolder
Evaluate newly activated prices against threshold and radius	FuelPrice, FuelStation, Location
Trigger and own notifications when a match occurs, respecting quiet hours	AlertNotification
Pause / resume / delete on the holder's request	RegisteredAccountHolder
Know its lifecycle status (active / paused)	—
Attributes • alertId : String • fuelCode : String • thresholdPrice : Float • radiusKm : Float • quietHours : TimeRange • status : String (active / paused)	
Relationships Generalisation: none Association: RegisteredAccountHolder (0..* savedAlerts to 1 holder); FuelPrice (0..* matchedAlerts to 0..* triggeringPrices) Aggregation: none Composition: whole of AlertNotification (1 ♦ 0..*) — notifications cannot exist without their alert	
Business Rules • Only a RegisteredAccountHolder can create an alert — guest users must register first. • An alert triggers only when an Active FuelPrice for its fuelCode, within radiusKm, falls to or below thresholdPrice. • No notifications during quietHours — matches are deferred and sent once quiet hours end.	

Table 10.2. CRC card for the new PriceAlert class: responsibilities, collaborators, attributes, relationships and business rules.

Class: AlertNotification (new class)	
Superclass: none Sub-class: none	
Responsibilities	Collaborators
Know its channel, send time and delivery status	—
Compose the alert message (station, fuel, new price, saving)	PriceAlert, FuelPrice, FuelStation
Deliver the message through the chosen channel (push / email)	Notification Service (push provider)
Confirm delivery, or retry when delivery fails	PriceAlert
Record the triggering price so the holder can verify the saving	FuelPrice
Attributes	

• notificationId : String • channel : String (push / email) • sentAt : DateTime • deliveryStatus : String (pending / delivered / failed)
Relationships Generalisation: none Association: none (reaches FuelPrice and the holder via its owning PriceAlert) Aggregation: none Composition: part of PriceAlert (alert 1 ♦ 0..* notifications) — deleted when its alert is deleted
Business Rules • A notification is created only by a triggered PriceAlert — never directly by a user or another class. • At most one notification per alert per price-drop event. • Failed deliveries are retried up to 3 times; after that deliveryStatus = failed and the holder is notified in-app.

Table 10.3. CRC card for the new AlertNotification class: responsibilities, collaborators, attributes and the composition relationship to its owning PriceAlert.

Tables 10.2 and 10.3 set out the CRC cards for PriceAlert and AlertNotification in the same plain-language form used for the legacy classes in Section 6. Read alongside the improved class diagram, the cards confirm the division of labour: PriceAlert holds the criteria and decides when a match has occurred, collaborating with FuelPrice, FuelStation and Location to test the threshold and radius; AlertNotification handles composing and delivering the message, collaborating with the Notification Service provider and reporting delivery back to its alert. Together they show, in a form a non-programmer can read, exactly how the new feature behaves and how it leans on the existing classes.

10.5 How the improvement adds value, in plain terms

Brought together, the improved diagrams tell a simple story. A registered motorist sets an alert (improved use case diagram and expanded use case); the system stores it as a PriceAlert linked to their account (improved class diagram and CRC cards); the Scheduler checks saved alerts whenever a new price becomes Active (reusing the state machine's existing event); and when a station within range drops to the target, an AlertNotification is sent through the push provider. No part of the legacy system is torn out — the improvement reuses the motorist's account, the location service, the price data and the daily cap. The result is a feature that is cheap to build because it is additive, and valuable because it converts the platform's existing data into money saved for citizens and measurably higher engagement for the service.

11. Limitations and Future Work

This report is bounded in several honest ways. As stated at the outset, it analyses Servo Saver from the customer (motorist) perspective only; the fuel-retailer, Service Victoria administrator and Consumer Affairs Victoria roles appear in the models for context but are not analysed at the same depth, and a fuller project would expand each into its own set of diagrams. The report is also the work of a single author modelling a system that, in reality, was built by a large team, so the models are a faithful but necessarily simplified reconstruction based on observed behaviour rather than internal documentation. The proposed Price Drop Alerts feature is specified at the analysis level — use cases, classes and behaviour — and would require detailed design, security review (particularly around storing user location preferences) and user testing before implementation. Natural extensions for future work include a favourites and watchlist capability feeding the alerts, per-station freshness indicators across all screens, and, in the longer term, coverage of electric-vehicle charging prices as the vehicle fleet changes.

12. Conclusion

Servo Saver is a well-conceived public service that makes a volatile, previously opaque market transparent for Victorian motorists. The UML deconstruction in this report shows a system whose central search journey is fast and resilient (the activity diagram), whose services are clearly defined for each actor (the use case diagrams), whose structure cleanly separates stations, prices, types and caps (the class diagram and CRC cards), and whose published prices are trustworthy because each one is validated, kept current and retained under a disciplined lifecycle (the state machine). The interface evaluation found real strengths in the map-first design and the forward-looking price cap, alongside a defining weakness — the service only works when the motorist remembers to use it — and a few presentation defects that erode trust. The proposed Price Drop Alerts feature addresses the core weakness directly by making the service proactive, and the improved use case and class models show that it integrates cleanly with the existing system, reusing the account, location, price and cap components already in place. Supported by evidence that proactive notifications sharply increase engagement and that timing fuel purchases delivers real savings, the improvement offers strong value for a modest, additive build — turning a capable lookup tool into an assistant that actively helps citizens pay less for fuel.

References

Airship. (2024). The 2024 mobile app retention benchmarks: How push notifications impact retention [Benchmark report]. Airship. <https://www.airship.com>

Australian Competition and Consumer Commission. (2024). There are tools available to save money on fuel [Media release]. ACCC. <https://www.accc.gov.au/media-release/there-are-tools-available-to-save-money-on-fuel>

Consumer Affairs Victoria. (2025). Mandatory fuel price reporting: Obligations for retailers. State Government of Victoria. <https://www.consumer.vic.gov.au/consumers-and-businesses/products-and-services/mandatory-fuel-price-reporting>

IBISWorld. (2025). Fuel retailing in Australia [Industry report]. IBISWorld. <https://www.ibisworld.com/australia/industry/fuel-retailing/438/>

Premier of Victoria. (2025). Daily fuel price cap now in place to stop price gouging [Media release]. State Government of Victoria. <https://www.premier.vic.gov.au/daily-fuel-price-cap-now-place-stop-price-gouging>

Service Victoria. (2025). Servo Saver: How the fuel price cap works. State Government of Victoria. <https://service.vic.gov.au/find-services/transport-and-driving/servo-saver>

Victorian Auditor-General's Office. (2020). Service Victoria — Digital delivery of government services. VAGO. <https://www.audit.vic.gov.au/report/service-victoria-digital-delivery-government-services>

Appendix

Appendix A — Additional interface captures

The following full-size captures supplement the compact versions used in the interface evaluation in Section 8 and the improvement in Section 9.

4:36 5G 34

Cancel Fuel type

☒ Unleaded 91 (U91)

☐ Premium Unleaded 95 (P95)

☐ Premium Unleaded 98 (P98)

☐ Diesel (DSL)

☐ Premium Diesel (PDSL)

☐ Ethanol 10 (E10)

☐ Ethanol 85 (E85)

☐ Biodiesel 20 (B20)

☐ Liquefied Petroleum Gas (LPG)

☐ Liquefied Natural Gas (LNG)

☐ Compressed Natural Gas (CNG)

Figure A.1. Fuel-type selector (full list of eleven fuel types).

4:36

5G 34

Cancel

Filter by brand

Reset

☐

7-Eleven

7-Eleven

☐

AAFCANS

AAFCANS

☐

Ampol

Ampol

☐

Apco Service Stations

Apco Service Stations

☐

APOLLO FUEL

APOLLO FUEL

☐

Arrow Petroleum

Arrow Petroleum

☐

ASTRON

ASTRON

☐

Atlas

ATLAS

☐

Auto Plus More

Auto Plus More

View results

Figure A.2. Brand filter (retailer checkbox list, showing the inconsistent logo placeholders).

Appendix B — Improved interface mock-up

Servo Saver — Improved Station Detail (proposed)

"After" mockup to pair with the live screenshot (Figure 5). Coral callouts mark the new / improved elements.

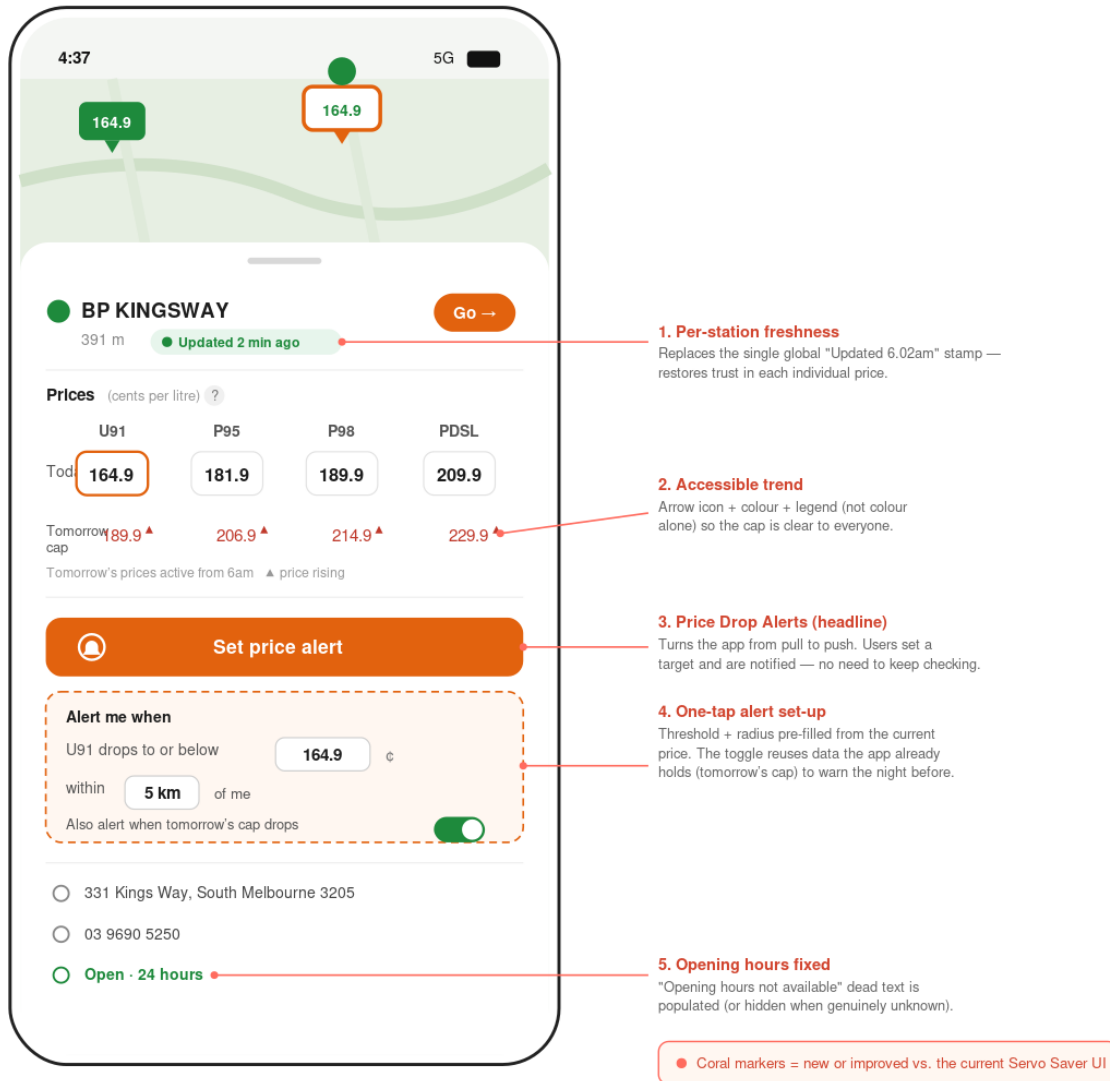


Figure B.1. Full proposed improved station-detail screen with annotated changes.

Appendix C — Note on submission

The cover page records placeholder fields for student ID and tutor name, which should be completed before submission. All diagrams in this report are reproduced from the author's final diagram set; the automatic table of contents updates its page numbers when the document is opened in Microsoft Word (right-click the table and choose "Update Field").